

Python Code

```
1  import numpy as np
2
3
4  def matrix_sum(matrix_1, matrix_2, coeff_1 = 1, coeff_2 = 2):
5      # to calc 2A - 3B
6
7      return coeff_1*matrix_1 + coeff_2*matrix_2
8
9
10 def matrix_product(matrix_1, matrix_2):
11     if matrix_1.shape[1] != matrix_2.shape[0]:
12         print("Shape mismatch.")
13         return 0
14     else:
15         matrix_12 = np.zeros((matrix_1.shape[0], matrix_2.shape[1]))
16         for i in range(matrix_12.shape[0]):
17             for j in range(matrix_12.shape[1]):
18                 for k in range(matrix_1.shape[1]):
19                     matrix_12[i, j] += matrix_1[i, k] * matrix_2[k, j]
20
21         return matrix_12
22
23
24 def matrix_transpose(matrix):
25     matrix_transpose = np.zeros((matrix.shape[1], matrix.shape[0]))
26
27     for i in range(matrix_transpose.shape[0]):
28         for j in range(matrix_transpose.shape[1]):
29             matrix_transpose[i, j] = matrix[j, i]
30
31     return matrix_transpose
32
33
34 def matrix_trace(matrix):
35     if matrix.shape[1] != matrix.shape[0]:
36         print("matrix has to be square")
37         return 0
38     else:
39         trace = 0
40         for i in range(matrix.shape[0]):
41             for j in range(matrix.shape[1]):
42                 if i==j:
43                     trace += matrix[i, j]
44
45     return trace
46
47
48 def minor(matrix, loc):
49     minor_mat = matrix.copy()
50     minor_mat = np.delete(minor_mat, loc[0], axis=0)
```

```

51     minor_mat = np.delete(minor_mat, loc[1], axis=1)
52
53     return minor_mat
54
55
56 def cofactor(matrix, loc):
57     minor_mat = minor(matrix, loc)
58     return (-1) ** (loc[0] + loc[1]) * matrix_determinant(minor_mat)
59
60
61 def matrix_determinant(matrix):
62     if matrix.shape[0] != matrix.shape[1]:
63         print("matrix has to be square")
64         return 0
65
66     if matrix.shape[0] == 1:
67         return matrix[0, 0]
68
69     if matrix.shape[0] == 2:
70         det = (
71             matrix[0, 0] * matrix[1, 1]
72             - matrix[0, 1] * matrix[1, 0]
73         )
74         return det
75
76     det = 0
77     for j in range(matrix.shape[1]):
78         det += matrix[0, j] * cofactor(matrix, (0, j))
79
80     return det
81
82
83 def inverse_by_adjoint(matrix):
84     if matrix.shape[0] != matrix.shape[1]:
85         print("matrix has to be square.")
86         return 0
87     else:
88         inverse_matrix = np.zeros_like(matrix, dtype=float)
89         det = matrix_determinant(matrix)
90         for i in range(matrix.shape[0]):
91             for j in range(matrix.shape[1]):
92                 inverse_matrix[i, j] = cofactor(matrix_transpose(matrix), [i,
93 j]) / det
94
95         return inverse_matrix
96
97
98
99 # =====
100
101
102 # Defining the matrices in question paper
103 matrix_A = np.array([[4, -2, 3], [1, 5, -1], [2, 0, 6]])

```

```
104 matrix_B = np.array([[ -1,  4,  2], [ 3, -2,  5], [ 0,  7, -3]])
105 matrix_C = np.array([[ 2,  1, -1], [ 3,  4,  2], [ 1, -2,  5]])
106 matrix_D = np.array([[ 1,  2,  0], [-1,  3,  4], [ 2, -2,  1]])
107
108
109
110 # Q1. 2A - 3B
111 print("2A - 3B= \n", matrix_sum(matrix_A, matrix_B, 2, -3), "\n")
112
113
114 # Q2. AB and BA
115 matrix_AB = matrix_product(matrix_A, matrix_B)
116 matrix_BA = matrix_product(matrix_B, matrix_A)
117 print("AB=\n", matrix_AB)
118 print("BA=\n", matrix_BA, "\n")
119
120
121 # Q3. Test of commutativity
122 if np.array_equal(matrix_AB, matrix_BA):
123     print("The multiplication of A and B is commutative.\n")
124 else:
125     print("The multiplication is not commutative for the given matrices.\n")
126
127
128 # Q4. A^T and B^T
129 print("A Transpose=\n", matrix_transpose(matrix_A))
130 print("B Transpose=\n", matrix_transpose(matrix_B), "\n")
131
132
133 # Q5. Transpose(AB) = Transpose(B)*Transpose(A)
134 AB_transpose = matrix_transpose(matrix_product(matrix_A, matrix_B))
135 B_transpose_A_transpose = matrix_product(matrix_transpose(matrix_B),
136 matrix_transpose(matrix_A))
137 print("Transpose of AB=\n", AB_transpose)
138 print("Transpose B times Transpose of A=\n", B_transpose_A_transpose)
139 if np.array_equal(AB_transpose, B_transpose_A_transpose):
140     print("The property holds.\n")
141 else:
142     print("The property does not hold. Check calculation.\n")
143
144
145 # Q6. Trace question
146 print("Trace of A= ", matrix_trace(matrix_A))
147 print("Trace of B= ", matrix_trace(matrix_B))
148 print("Trace of AB= ", matrix_trace(matrix_AB))
149 print("Trace of BA= ", matrix_trace(matrix_BA), "\n")
150
151 if matrix_trace(matrix_AB) == matrix_trace(matrix_BA):
152     print("trace(AB) = trace(BA) holds.\n")
153 else:
154     print("The property does not hold. Check calculation.\n")
155
156 # Q7. C+D and C-D
```

```
157 print("C+D= \n", matrix_sum(matrix_C, matrix_D, 1, 1), "\n")
158 print("C-D= \n", matrix_sum(matrix_C, matrix_D, 1, -1), "\n")
159
160
161 # Q8. Determinant of C
162 print("Determinant of C= ", matrix_determinant(matrix_C), "\n")
163 if matrix_determinant(matrix_C) == 0:
164     print("Matrix C is singular.\n")
165 else:
166     print("Matrix C is not singular.\n")
167
168
169 # Q9. Inverse of C
170 C_inverse = inverse_by_adjoint(matrix_C)
171 print("Inverse of C, calculated with adjoint method:\n", C_inverse, "\n")
172
173
174 # Q10. Verify with C*C_inverse
175 C_C_inverse = matrix_product(matrix_C, C_inverse)
176 print("C multiplied with C_inverse:\n", C_C_inverse, "\n")
177
178 identity_matrix = np.eye(matrix_C.shape[0])
179 if np.allclose(C_C_inverse, identity_matrix): # allclose to ignore very small
differences
180     print("Multiplication Verified.\n")
181 else:
182     print("Recheck calculation.\n")
183
184
185 # Q11. solve linear system
186 '''
187 2x + y - z = 7
188 3x + 4y + 2z = 16
189 x - 2y + 5z = 9
190 '''
191
192 print("The coeeficient matrix is same as C. We solve X by calculating
C_inverse times y.")
193 vector_y = np.array([[7], [16], [9]])
194 solution = matrix_product(C_inverse, vector_y)
195 print(f"Solution:\n x={solution[0, 0]},\n y={solution[1, 0]},\n z={solution[2,
0]},\n")
196
197
198 # Q12. compute CD
199 matrix_CD = matrix_product(matrix_C, matrix_D)
200 print("CD=\n", matrix_CD, "\n")
201
202 matrix_DC = matrix_product(matrix_D, matrix_C)
203
204 if np.array_equal(matrix_CD, matrix_DC):
205     print("CD = DC, so the order does not matter for these matrices.\n")
206 else:
207     print("CD != DC, so the order of multiplication must be preserved.\n")
```

```
208
209 print("The reason order matters is that matrix multiplication is based on row-
    by-column multiplication. Each element of the resulting matrix depends on a
    specific row from the first matrix and a specific column from the second
    matrix. When the order is reversed, the rows and columns being combined
    change, so the resulting values generally change.")
210
211
212
213
214 '''
215 Output of the code:
216 ^^^^^^^^^^^^^^^^^^^^^^^^^^^
217
218 2A - 3B=
219 [[ 11 -16  0]
220 [ -7  16 -17]
221 [  4 -21 21]]
222
223 AB=
224 [[-10.  41. -11.]
225 [ 14. -13. 30.]
226 [ -2.  50. -14.]]
227 BA=
228 [[  4.  22.  5.]
229 [ 20. -16. 41.]
230 [  1.  35. -25.]]
231
232 The multiplication is not commutative for the given matrices.
233
234 A Transpose=
235 [[ 4.  1.  2.]
236 [-2.  5.  0.]
237 [ 3. -1.  6.]]
238 B Transpose=
239 [[-1.  3.  0.]
240 [ 4. -2.  7.]
241 [ 2.  5. -3.]]
242
243 Transpose of AB=
244 [[-10.  14. -2.]
245 [ 41. -13. 50.]
246 [-11.  30. -14.]]
247 Transpose B times Transpose of A=
248 [[-10.  14. -2.]
249 [ 41. -13. 50.]
250 [-11.  30. -14.]]
251 The property holds.
252
253 Trace of A= 15
254 Trace of B= -6
255 Trace of AB= -37.0
256 Trace of BA= -37.0
257
```

```
258 trace(AB) = trace(BA) holds.
259
260 C+D=
261 [[ 3  3 -1]
262  [ 2  7  6]
263  [ 3 -4  6]]
264
265 C-D=
266 [[ 1 -1 -1]
267  [ 4  1 -2]
268  [-1  0  4]]
269
270 Determinant of C= 45
271
272 Matrix C is not singular.
273
274 Inverse of C, calculated with adjoint method:
275 [[ 0.53333333 -0.06666667  0.13333333]
276  [-0.28888889  0.24444444 -0.15555556]
277  [-0.22222222  0.11111111  0.11111111]]
278
279 C multiplied with C_inverse:
280 [[1.00000000e+00 0.00000000e+00 0.00000000e+00]
281  [2.22044605e-16 1.00000000e+00 0.00000000e+00]
282  [0.00000000e+00 0.00000000e+00 1.00000000e+00]]
283
284 Multiplication Verified.
285
286 The coefficient matrix is same as C. We solve X by calculating C_inverse times
287 y.
288 Solution:
289 x=3.866666666666667,
290 y=0.48888888888888893,
291 z=1.2222222222222223,
292
293 CD=
294 [[ -1.   9.   3.]
295  [  3.  14.  18.]
296  [ 13. -14. -3.]]
297
298 CD != DC, so the order of multiplication must be preserved.
299
300 The reason order matters is that matrix multiplication is based on row-by-
301 column multiplication. Each element of the resulting matrix depends on a
302 specific row from the first matrix and a specific column from the second
303 matrix. When the order is reversed, the rows and columns being combined
304 change, so the resulting values generally change.
```